

Signal Clone

A secure-messaging platform reproducing Signal's layout, conversation model and real-time behaviour. Encryption is simulated. Everything else is real, persisted and wired end to end.



FRONTEND

Next.js 16, TypeScript, Tailwind v4

BACKEND

FastAPI, SQLAlchemy 2.0, Alembic

DATABASE

SQLite, WAL mode, FTS5

REAL TIME

Native WebSocket, JWT handshake

SCOPE

13 phases, ~24 hours

DOCUMENT DATE

8 October 2026

This document is a walkthrough, not a specification. It is written so the reader can explain every decision in the project to someone who has not seen the code: what was built, why that option was chosen over the alternative, and how to verify each claim.

What was asked for, and what that actually means

The assignment asks for a functional clone of Signal: registration, contacts, one-to-one and group conversations, and real-time messaging, inside an interface that looks and behaves like the real app.

The required feature set

AREA	WHAT MUST WORK
Authentication	Register by phone number or username with a mocked verification code, set a display name and avatar, sign in and out, and keep the session across a refresh.
Conversation list	Threads sorted by most recent activity, search across conversations and contacts, add a contact, unread badges with a last-message preview, and online or last-seen state.
Direct messaging	Real-time send and receive, timestamps, delivery and read receipts as the single and double check experience, typing indicators, the four message statuses, and persistence of everything.
Group messaging	Create a group with a name and members, send and receive, view the member list, add and remove members under admin control, and persist all of it.
Signal experience	Matching navigation, bubbles and threading, forms, modals, search and filters, toasts, and settings placeholders.

What may be mocked

The brief explicitly permits simulating four things: voice and video calls, stories, linked devices, and end-to-end encryption. These may be present as Coming Soon placeholders.

How it is graded

CRITERION	WHAT THE REVIEWER LOOKS FOR
Functionality	All core features working, including real-time messaging
UI and UX	Visual similarity to the original app
Database design	Well-structured schema with proper relationships
Backend and API design	Clean, sensible architecture
Code quality	Readable and well organised
Code modularity	Separation of concerns, reusable components
Code understanding	Ability to explain the implementation in an interview

The reading that shaped every decision below. Three of the seven criteria are about how the code is organised and explained rather than what it does. That makes the schema, the module boundaries and this document part of the deliverable, not overhead on top of it.

Every choice, and the alternative it beat

These are the questions a reviewer is most likely to ask. Each answer names the option that was rejected, because a decision with no discarded alternative is not a decision.

Why FastAPI rather than Django?

The brief allows either. Django would need Channels plus a channel layer, which in practice means Redis, to hold a WebSocket. FastAPI speaks WebSocket natively in the same process, so real-time messaging needs no broker and the whole thing deploys as one service. Django's advantages here, the admin and the auth system, are not useful when authentication is a mocked OTP flow and the data is seeded by script.

Why SQLAlchemy 2.0 with Alembic rather than raw SQL?

Database design is a graded criterion, and Alembic turns the schema into reviewable history rather than one generated file. The 2.0 typed ORM also means relationships are declared once and the typing follows through into the service layer.

Why is a conversation one table instead of separate direct and group tables?

Because almost everything about a thread is identical whichever it is: it has members, messages, a last-activity timestamp, read cursors, mute and pin state. Splitting them would duplicate all of that and force every query to union two tables. A `type` column distinguishes them, and a sorted-pair `dm_key` with a unique index enforces the one rule that differs, which is that two people may only have one direct thread.

Why does read state live on the membership instead of the message?

Storing a read flag per message per member would mean writing one row per recipient every time someone opens a thread. Instead each membership holds a cursor, `last_read_message_id`, and the unread count is derived by counting messages after it. Opening a thread is then a single-row update regardless of how many messages were unread.

Then why does a separate receipts table exist?

Because the sender's check marks need per-recipient truth, and in a group one message is delivered many times. The cursor answers "what has this person read", which is what the badge needs. The receipts table answers "who has received and read this specific message", which is what the check marks need. They are different questions and conflating them produces wrong ticks in groups.

How do check marks work in a group, then?

They roll up. Delivered means every currently active member has a delivery row. Read means every active member has a read row. The `status` column on the message caches that rollup so the thread renders without an aggregate query per bubble.

What stops a duplicate message when the network is flaky?

The client generates a `client_id` before sending and the pair of sender and client id carries a unique index. A retry therefore hits the same row rather than creating a second one, and the optimistic bubble in the UI reconciles on the same key.

Why cursor pagination rather than offset?

An offset shifts underneath you. If a message arrives while someone is scrolling up through history, page two computed by offset either repeats or skips a row. A cursor anchored to a message id is stable regardless of what lands.

Where does the access token live, and why not localStorage?

In memory in the client module. A token in localStorage is readable by any script that gets injected into the page. The refresh token is an httpOnly cookie, which script cannot read at all, and it is stored server-side as a hash so logout can genuinely revoke it rather than just forgetting it.

What exactly is mocked about the encryption?

All of the cryptography. There is no X3DH key agreement and no Double Ratchet. What exists is the surface: each account carries a random identity key and registration id, each message carries a derived envelope hash, the thread header shows the encryption notice, and the contact sheet exposes a safety-number screen built from both parties' identity keys. Message bodies are stored as plaintext and the README says so rather than implying otherwise.

Why is presence real rather than random?

The brief allows mocking it, but the WebSocket hub already knows exactly who is connected. Writing `is_online` on connect and `last_seen_at` on disconnect is less code than faking it and it is actually correct.

How the pieces fit

Two deployables. The browser talks to FastAPI over HTTPS for everything that must survive a refresh, and holds one WebSocket open for everything that must arrive within a frame. The socket never grants authority it did not get from a REST call first.

Repository layout

```

signal-clone/
├── backend/
│   ├── app/
│   │   ├── api/v1/      routers, one module per feature area
│   │   ├── core/       settings, security, shared dependencies
│   │   ├── db/         engine, session, base, seed script
│   │   ├── models/     SQLAlchemy tables
│   │   ├── schemas/    Pydantic request and response models
│   │   ├── services/   business rules, kept out of the routers
│   │   ├── realtime/   connection hub and event definitions
│   │   └── main.py     application factory
│   ├── alembic/        migrations
│   ├── tests/
│   ├── media/          uploaded files, gitignored
│   └── requirements.txt
└── frontend/
    └── src/
        ├── app/        App Router pages
        ├── components/  UI, one file per component
        ├── hooks/       useSocket, useConversation, useTyping
        ├── lib/         API client, config, shared types
        └── store/       Zustand slices

```

The layering rule

Routers parse and validate, then call a service. Services hold the business rules and are the only layer that touches the ORM. The realtime hub is a transport, not a decision maker: it broadcasts what a service already committed. This is what keeps the modularity criterion satisfiable, and it is also why a router file never runs more than a few lines.

The life of a message

1. The composer appends an optimistic bubble keyed by a client-generated `client_id`, at status **sending**.
2. A POST persists the row and returns the server id. The optimistic bubble reconciles on `client_id`, so a retry cannot duplicate it. Status becomes **sent**.
3. The hub fans a `message.new` event to every connected member except the sender, and a `message.status` event back to the sender.
4. Each recipient's socket writes a receipt row on arrival. Once every active member has one, the sender's bubble becomes **delivered**.
5. Opening the thread posts a read cursor, which fills `read_at` on the receipts. Once every active member has read, the bubble becomes **read**.

Why the order matters. Persistence happens before broadcast, never alongside it. If the write fails, nothing was announced. If the broadcast fails, the message is still in the database and the recipient picks it up on reconnect through replay-by-cursor. There is no state that exists only in a socket.

Reconnection

The client heartbeats every twenty-five seconds and reconnects with exponential backoff. On resume it sends the id of the last message it saw per open thread, and the server replays anything newer. That is why a dropped connection is invisible to the user rather than a reason to refresh.

Eleven tables

Three decisions drive the shape: a conversation is one object whether it holds two people or twenty, read state belongs to the membership rather than the message, and a receipt is per recipient because a group message is delivered many times.

Relationships

PARENT	CARD .	CHILD
users	1:N	contacts (owner side), contacts (target side), conversation_members, messages, devices, auth_sessions
conversations	1:N	conversation_members, messages
messages	1:N	message_receipts, reactions, attachments, messages (reply_to, self join)

phone_verifications stands alone with no foreign key, because a code is requested before the account exists.

users One row per account: identity, profile, presence.

COLUMN	TYPE	NOTES
id	TEXT PK	UUID4 string, so ids never leak a row count
phone_number	TEXT UNIQUE	E.164. The primary sign-in handle
username	TEXT UNIQUE NULL	Optional handle, discovery without sharing a number
display_name	TEXT	Set during onboarding
about	TEXT NULL	Signal's short status line
avatar_url	TEXT NULL	Null falls back to initials on avatar_color
avatar_color	TEXT	One of Signal's twelve swatch keys
identity_key	TEXT	Mocked public key, feeds the safety-number screen
registration_id	INTEGER	Mocked device registration id
is_online	BOOLEAN	Flipped by the hub on connect and disconnect
last_seen_at	DATETIME	Written on disconnect, drives the last-seen line
created_at / updated_at	DATETIME	

conversations One object for direct and group threads alike.

COLUMN	TYPE	NOTES
id	TEXT PK	
type	TEXT	direct or group, constrained rather than free text
name	TEXT NULL	Groups only. A direct thread takes its name from the peer
description	TEXT NULL	Group description
avatar_url	TEXT NULL	
avatar_color	TEXT	Same swatch set as users
dm_key	TEXT UNIQUE NULL	Sorted pair of account ids. Enforces one direct thread per pair
created_by	TEXT FK users	Group creator, the first admin
disappearing_seconds	INTEGER	Zero means off. Applies to new messages only
last_message_id	TEXT FK NULL	Denormalised so the list renders without a join per row
last_activity_at	DATETIME	The sort key for the conversation list
created_at / updated_at	DATETIME	

conversation_members Membership plus everything that is per person rather than per thread.

COLUMN	TYPE	NOTES
id	TEXT PK	
conversation_id	TEXT FK	
user_id	TEXT FK	
role	TEXT	admin or member. Gates the add and remove endpoints
joined_at	DATETIME	
left_at	DATETIME NULL	Soft leave, so history stays attributable
last_read_message_id	TEXT FK NULL	The read cursor. Unread count is derived from it
muted_until	DATETIME NULL	
is_pinned / is_archived	BOOLEAN	Per person, not per thread

messages The thread itself. Soft deleted, never hard deleted.

COLUMN	TYPE	NOTES
id	TEXT PK	
conversation_id	TEXT FK	
sender_id	TEXT FK NULL	Null for system events such as a member joining
type	TEXT	text, image, file or system
body	TEXT NULL	Plaintext. The simulation does not pretend otherwise at rest
envelope_hash	TEXT	Derived digest, shown on the message-info sheet
reply_to_id	TEXT FK NULL	Self join behind the quoted strip
status	TEXT	Sender-side rollup: sending, sent, delivered, read
client_id	TEXT	Supplied by the sender. Makes a resend idempotent
edited_at	DATETIME NULL	Non-null renders the edited marker
deleted_at	DATETIME NULL	Non-null renders the tombstone bubble
expires_at	DATETIME NULL	Set when the thread has a timer. Swept server-side
created_at	DATETIME	Sort key within the thread

message_receipts Per-recipient delivery truth. The source for check marks.

COLUMN	TYPE	NOTES
id	TEXT PK	
message_id	TEXT FK	
user_id	TEXT FK	The recipient, never the sender
delivered_at	DATETIME NULL	Written when a socket acknowledges arrival
read_at	DATETIME NULL	Written when the read cursor passes this message

contacts Directed address book, a self-join on users.

COLUMN	TYPE	NOTES
id	TEXT PK	
owner_id	TEXT FK users	Who holds this address book
contact_user_id	TEXT FK users	Who is in it
nickname	TEXT NULL	Local override, never touches the other account
is_blocked	BOOLEAN	Blocks delivery in both directions
created_at	DATETIME	

reactions One emoji per person per message, matching Signal.

COLUMN	TYPE	NOTES
id	TEXT PK	
message_id	TEXT FK	
user_id	TEXT FK	
emoji	TEXT	Replacing a reaction updates this row in place
created_at	DATETIME	

attachments Files and images hung off a message.

COLUMN	TYPE	NOTES
id	TEXT PK	
message_id	TEXT FK NULL	Null while uploaded but not yet sent
uploader_id	TEXT FK users	
file_name / content_type / size_bytes	TEXT, TEXT, INT	
storage_path	TEXT	Local disk in development, object storage in production
width / height	INTEGER NULL	Lets the bubble reserve space before the image loads
thumbnail_path	TEXT NULL	

devices · auth_sessions · phone_verifications Supporting tables.

TABLE	COLUMNS AND PURPOSE
devices	id, user_id, name, platform, is_primary, last_active_at. Backs the Linked Devices screen with real rows instead of a static mock.
auth_sessions	id, user_id, refresh_token_hash, user_agent, expires_at, revoked_at. Hashed, so a database dump does not hand over live sessions. Logout stamps revoked_at rather than deleting.
phone_verifications	id, phone_number, code, attempts, expires_at, consumed_at. The mocked OTP modelled as if it were real, including single use and an attempt counter.

Indexes and constraints

PURPOSE	DEFINITION
Thread paging	messages (conversation_id, created_at DESC)
Idempotent send	UNIQUE messages (sender_id, client_id)
Conversation list sort	conversations (last_activity_at DESC)
One direct thread per pair	UNIQUE conversations (dm_key)
One membership per person	UNIQUE conversation_members (conversation_id, user_id)
One receipt per recipient	UNIQUE message_receipts (message_id, user_id)
One reaction per person	UNIQUE reactions (message_id, user_id)

One address-book entry	UNIQUE contacts (owner_id, contact_user_id)
Disappearing sweeper	messages (expires_at) WHERE expires_at IS NOT NULL
Message search	FTS5 virtual table over messages.body, synced by trigger
SQLite pragmas	journal_mode=WAL, foreign_keys=ON, busy_timeout=5000

WAL matters because the default rollback journal blocks readers during a write. With a WebSocket hub broadcasting while messages are being inserted, that would be felt.

REST for durability, one socket for immediacy

Everything that must survive a refresh goes through REST under the versioned prefix. Everything that must arrive within a frame goes through the socket. Nothing is authorised by the socket alone.

Authentication

VERB	PATH	PURPOSE
POST	/auth/request-code	Creates a verification row, returns the code in development
POST	/auth/verify	Exchanges a code for tokens, flags whether the account is new
POST	/auth/register	Completes the profile: display name, username, avatar
POST	/auth/refresh	Rotates the refresh token, reissues an access token
POST	/auth/logout	Revokes the current session row
GET	/auth/me	Rehydrates the session on page load

Profile and contacts

VERB	PATH	PURPOSE
PATCH	/users/me	Display name, about line, avatar colour
POST	/users/me/avatar	Multipart upload
GET	/users/search	Lookup by phone number or username
GET	/contacts	Address book with presence folded in
POST	/contacts	Adds by phone number or username
PATCH	/contacts/{id}	Nickname and block state
DELETE	/contacts/{id}	Removes the entry, keeps the conversation

Conversations

VERB	PATH	PURPOSE
GET	/conversations	Sorted by last activity, filters for unread and archived
POST	/conversations	Direct by peer id, or group by name and member list
GET	/conversations/{id}	Detail with members, roles and thread settings
PATCH	/conversations/{id}	Name, description, avatar, disappearing timer
POST	/conversations/{id}/read	Moves the read cursor, clears the badge
PATCH	/conversations/{id}/prefs	Pin, mute, archive, scoped to the caller
POST	/conversations/{id}/members	Admin only. Emits a system message

PATCH	/conversations/{id}/members/{uid}	Admin only. Promote or demote
DELETE	/conversations/{id}/members/{uid}	Admin only. Stamps left_at
POST	/conversations/{id}/leave	Self removal, reassigns admin if the last one leaves

Messages

VERB	PATH	PURPOSE
GET	/conversations/{id}/messages	Cursor paged backwards from a message id
POST	/conversations/{id}/messages	Body, client id, optional reply target and attachments
PATCH	/messages/{id}	Edit within a window, stamps edited_at
DELETE	/messages/{id}	Soft delete, renders as a tombstone for everyone
PUT	/messages/{id}/reaction	Sets or replaces the caller's emoji
GET	/messages/search	FTS5 across every thread the caller belongs to
POST	/attachments	Multipart upload ahead of send

WebSocket events

DIRECTION	EVENTS
Client to server	typing.start, typing.stop, message.delivered, presence.ping
Server, messages	message.new, message.updated, message.deleted, message.status
Server, threads	conversation.created, conversation.updated, member.added, member.removed
Server, people	typing, presence.changed, reaction.updated
Transport	Heartbeat every 25s, exponential reconnect, replay missed messages by cursor

The hub keeps a map of account id to a set of live sockets, so one account open in several tabs behaves the way Signal behaves across several devices.

Signal's values, held as tokens

Every colour and metric lives as a CSS variable mapped into the Tailwind theme. No component hard-codes a colour, which is what makes the dark theme a token swap rather than a second set of classes.

Palette

TOKEN	LIGHT	DARK
ultramarine	#3a76f0	#4b81f2
surface	#ffffff	#1b1b1b
surface-raised	#f6f6f6	#121212
border	#e0e0e0	#3b3b3b
ink	#1b1b1b	#f0f0f0
ink-2	#6b6b6b	#b9b9b9
bubble-in	#e9e9e9	#2e2e2e
bubble-out	#3a76f0	#3a76f0

Twelve avatar swatches are defined as well, from **a100** crimson through **a210** ultramarine. Signal assigns one deterministically per contact, which is why a conversation list of people without photos still reads as distinct rows.

Metrics taken from Signal Desktop

Typeface	Inter, the face Signal Desktop ships
Navigation rail	68px, icon only, avatar and settings pinned to the bottom
Conversation list	320px default, resizable between 280 and 380
Thread width	Capped at 720px and centred
Row height	72px per conversation row
Avatars	48px in the list, 36px in a group header, 28px inline
Bubble radius	18px, dropping to 4px on the tail corner of a grouped run
Composer	Pill at 24px radius, mic becomes a send arrow once there is text
Type scale	15px body, 14px names, 13px previews, 11px timestamps
Check marks	One outline sent, two outline delivered, two filled read
Breakpoint	Single pane under 768px, the list slides over the thread

Component inventory

The whole product decomposes into these, one file each. This is the modularity criterion made concrete.

- **Shell:** NavRail, ConversationListPane, ChatPane, EmptyState
- **List:** ConversationRow, UnreadBadge, SearchField, FilterTabs, ArchiveRow

- **Thread:** MessageList, MessageBubble, QuotedReply, DateDivider, SystemMessage, TypingIndicator, StatusTicks, ReactionBar, ReactionPills
- **Composer:** Composer, AttachmentTray, EmojiPicker, DisappearingBadge
- **Header:** ChatHeader, PresenceLine, CallButtons, ThreadMenu
- **Panels:** ContactSheet, GroupInfoPanel, MemberRow, SafetyNumberView
- **Modals:** NewChatModal, NewGroupModal, AddMembersModal, ConfirmDialog
- **Settings:** SettingsLayout and one section per area
- **Primitives:** Avatar, Toast, Tooltip, Spinner, IconButton, Sheet

Phase 0, and how it was verified

The scaffold is complete and committed. Every phase in this project carries a gate, and a gate is something observed rather than asserted.

What exists in the backend

- An application factory in `main.py` that mounts CORS, the media directory, the versioned router and a health probe, and does nothing else. Routing, rules and transport each live elsewhere.
- A single settings object built on pydantic-settings, with development-safe defaults for every value, so a fresh clone runs with no `.env` file. Nothing in the codebase reads the environment directly.
- Package structure already carved for models, schemas, services, database and realtime, so later phases add files rather than reorganising.

What exists in the frontend

- The Signal palette and metrics as CSS variables in `globals.css`, mapped into the Tailwind theme so components write `bg-surface` or `text-ink-2`.
- The dark theme defined twice on purpose: once behind a `data-theme` attribute for the settings toggle, once behind `prefers-color-scheme` for the operating system default. The toggle can therefore override the OS in both directions.
- A typed fetch client that holds the access token in memory, sends credentials so the refresh cookie travels, and flattens FastAPI's error shape into one `ApiError` type with a usable message.
- A temporary index route that probes both API endpoints, so the phase gate is visible in a browser.

Verification evidence

CHECK	RESULT
Backend imports, route table enumerated	5 routes registered
Frontend TypeScript compile	Clean, no errors
<code>GET /health</code>	200, correct JSON body
<code>GET /api/v1/ping</code>	200, correct JSON body
CORS preflight from port 3000	Allowed, with credentials
Next.js page render	200

One deviation from the plan, worth knowing about. The Next.js generator produced version 16 rather than 15, and version 16 enables `cacheComponents` by default. That changes how data may be fetched during server rendering, so the live thread will be built as client components with Suspense boundaries in the shell. No feature is affected.

Thirteen phases, each with a gate

Ordered so the product becomes demonstrable early and stays that way. The backend leads, because a typed client generated from a settled schema removes a whole class of rework. If a gate does not pass, the next phase does not start.

-
- 00 Scaffold** DONE
- GATE** Both dev servers start and the browser reaches the health route through the API client.
-
- 01 Data layer** NEXT
- GATE** The seed runs on a fresh database and a SQL client shows populated threads with sensible timestamps.
-
- 02 Authentication**
- GATE** A full signup and a returning sign-in both work, and refresh survives an expired access token.
-
- 03 Conversations and messages**
- GATE** Every core endpoint is exercised from the OpenAPI page and the rows look right in the database.
-
- 04 Realtime hub**
- GATE** Two tabs on different accounts exchange messages with no refresh, and a dropped socket recovers on its own.
-
- 05 Frontend shell**
- GATE** The shell matches a Signal screenshot side by side at desktop and phone widths.
-
- 06 Onboarding**
- GATE** A reviewer reaches the chat list from a cold browser in under fifteen seconds.
-
- 07 Conversation list**
- GATE** The list reorders live when a message lands in a thread that is not open.
-
- 08 Chat pane**
- GATE** All four message statuses are observable in a two-tab demo, and nothing duplicates on a flaky send.
-
- 09 Groups**
- GATE** A non-admin sees the controls absent, and the server refuses the call if they forge it anyway.
-
- 10 Signal surface**
- GATE** Every navigation target resolves to a real screen, with no dead links.
-
- 11 Bonus features**
- GATE** Each bonus item either works end to end or is removed. Nothing ships half-wired.
-
- 12 Documentation and deploy**
- GATE** A stranger can follow the README from clone to running locally, and the hosted link works from a phone.
-

Known risks, stated rather than discovered later

SQLite on a free host is not durable

Render's free tier has an ephemeral filesystem, so the database resets on redeploy. Either attach a persistent disk on the paid tier, or accept the reset and re-seed on boot. Whichever is chosen gets written into the README rather than left for the reviewer to find.

Vercel cannot hold a WebSocket

Serverless functions time out, so the socket must terminate on the backend service. The frontend therefore talks to two origins, which makes CORS and secure-socket configuration load-bearing rather than incidental.

Pixel fidelity has no upper bound

The brief asks for an exact match, which is not achievable in the time budget. Layout, spacing, colour and interaction behaviour are matched; small iconography differences are accepted. Phase 5 includes an explicit side-by-side comparison so the gap is measured rather than guessed.

From a clone to a running app

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # macOS and Linux
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

The API is then on port 8000, with interactive documentation at `/docs`. No `.env` file is needed locally; copy `.env.example` only to override something.

Frontend

```
cd frontend
npm install
npm run dev
```

The app is then on port 3000.

Assumptions a reader should know

- Phone verification is mocked with a fixed code, and in development that code is returned in the response body so a reviewer never has to guess it.
- Encryption is simulated. There is no key agreement and no ratchet. Message bodies are plaintext at rest and the documentation says so.
- Online and last-seen state is driven by real socket connect and disconnect, not a random generator.
- Everything is written from scratch for this assignment. No existing clone repository was used as a source, and no code was taken from the official Signal clients, which are GPL-licensed.